Python Engineering - Test Runner Function References

TESTS, test vs test(), PASS / FAIL / ERROR, and exception hierarchy

1. Core idea: a function is also an object

In Python, a function is not only something we can execute. It is also an object that can be stored in a variable, placed inside a list, or passed as an argument.

```
def test_initial_state():
    ...

function_reference = test_initial_state
# no parentheses: the function is NOT executed yet
```

function_reference points to the function. We have not called it yet.

2. Difference between test and test()

Expression	Meaning
test	Reference to the function. It is not executed.
test()	Function call. Execute it now.

```
test_initial_state    # function object / reference
test_initial_state()  # execute the function
```

Parentheses are the call operator. This is the same distinction previously seen between add and add(...).

3. Why create a TESTS list?

```
TESTS = [
    test_initial_state,
    test_euler_step,
    test_csv_round_trip,
    test_missing_file_raises,
]
```

We store function references so the runner can treat every test in the same generic way. The list stores functions to be executed later, not their results.

This lets one algorithm execute any number of tests.

4. What if we use parentheses inside TESTS?

```
# Not what we want here:
TESTS = [
    test_initial_state(),
    test_euler_step(),
]
```

The functions would execute immediately while Python builds the list. The list would usually contain their return values, often None, rather than function references.

5. What does for test in TESTS mean?

```
for test in TESTS:
    ...
```

On each iteration, the local variable test refers to a different function.

```
iteration 1: test = test_initial_state
iteration 2: test = test_euler_step
iteration 3: test = test_csv_round_trip
iteration 4: test = test_missing_file_raises
```

The name test is simply the loop variable. It communicates intent but has no special Python meaning.

6. Why write test() inside try?

```
for test in TESTS:
    try:
        test()
```

At that point we want to execute whichever function test currently references.

```
test = test_initial_state
test()
# equivalent to:
test_initial_state()
```

On the next iteration the exact same test() line calls another function. The runner does not need to know its internal details.

7. Why this pattern scales

```
def test_specific_energy():
    ...

TESTS = [
    test_initial_state,
    test_euler_step,
    test_csv_round_trip,
    test_missing_file_raises,
    test_specific_energy,
]
```

The loop and PASS / FAIL / ERROR logic do not change.

8. Functions as objects - connection to Class 10

```
TESTS = [function_A, function_B, function_C]

for function in TESTS:
    function()
```

The runner treats functions as data: store them, iterate over them, then call them.

9. Overall test-runner flow

```
for test in TESTS:
    try:
        test()
    except AssertionError as error:
        # FAIL
        ...
    except Exception as error:
        # ERROR
        ...
    else:
        # PASS
        ...
```

There are three conceptually different outcomes: PASS, FAIL and ERROR.

10. PASS

If test() finishes without raising an exception, the else block runs.

```
test() -> no exception -> else -> PASS
```

PASS means the test executed and all its assertions were satisfied.

11. FAIL: AssertionError

```
result = np.array([102.0, 19.019, 0.1])
expected = np.array([999.0, 19.019, 0.1])

assert np.allclose(result, expected)
# -> AssertionError
```

The test reached the check, but observed behavior did not match the expectation. The runner classifies this as FAIL.

12. ERROR: another unexpected exception

```
float('hello')        # ValueError
[10, 20][50]          # IndexError
10 / 0                # ZeroDivisionError
missing_dict['x']     # KeyError
```

The test could not complete its normal verification path, so the runner classifies it as ERROR.

13. Exception does not mean only 'bad content'

```
Exception
|-- ValueError
|-- TypeError
|-- FileNotFoundError
|-- IndexError
|-- KeyError
|-- ZeroDivisionError
`-- AssertionError
```

The broad except Exception therefore means approximately: if another normal exception reaches this boundary, report it as ERROR.

14. AssertionError is also an Exception

AssertionError belongs to the Exception family too.

```
AssertionError
    is an Exception
```

Python checks except blocks in order and uses the first compatible handler.

15. except order matters

```
except AssertionError as error:
    # specific -> FAIL
    ...

except Exception as error:
    # general -> ERROR
    ...
```

Specific exception first, general exception later.

If reversed:

```
except Exception as error:
    # catches AssertionError too
    ...

except AssertionError as error:
    # would never classify it here
    ...
```

An AssertionError would be classified as ERROR instead of FAIL.

16. General ordering rule

```
specific exception first
        |
        v
more general exception later
```

This is a general exception-handling rule, not only a testing rule.

17. Why use Exception here but not in EX23?

In EX23 we knew the specific failures we intended to handle:

```
FileNotFoundError
ValueError
```

So specific handlers were better.

At the outer boundary of run_tests.py, the goal is different: one unexpected test error should not prevent the remaining tests from running.

```
test 1 -> PASS
test 2 -> TypeError
test 3 -> should still run
test 4 -> should still run
```

That is why the runner uses a final broad handler.

18. Catching Exception is not hiding the error

```
except Exception as error:
    print(f'ERROR {test.__name__}')
    print(f'{type(error).__name__}: {error}')
```

The exception remains visible. We only prevent one test from terminating the entire runner.

19. Complete outcome classification

Situation	Exception	Classification
All assertions are True	None	PASS
assert result == expected is False	AssertionError	FAIL
float('hello')	ValueError	ERROR
invalid index access	IndexError	ERROR
unexpected missing file	FileNotFoundError	ERROR

20. Final summary

```
TESTS
-> list of function references

for test in TESTS
-> test refers to a different function
   on each iteration

test
-> function reference

test()
-> execute that function

AssertionError
-> FAIL
-> test reached its check,
   but the expectation was not satisfied

another Exception
-> ERROR
-> something unexpected happened
   while executing the test

no exception
-> PASS
```

Core idea: TESTS lets us treat functions as data; test() executes them, and exception handling classifies each execution outcome.